

Documentacion Final - Proyecto 4

VeriLang en Kotlin: ejecucion del lenguaje Rascal desde una aplicacion Kotlin

Campo	Informacion
Curso	ISIS-2111 - Elementos Esenciales de Lenguajes de Programacion
Estudiante	Ronny Esteban Lopez Martinez
Codigo	202415741
Proyecto	Proyecto 4 - VeriLang ejecutado desde Kotlin
Raiz del proyecto	proyecto_rascal
Contenido de la entrega	Proyecto Rascal, aplicacion Kotlin, jar de ejecucion Rascal, casos de prueba y documento explicativo final

Resumen ejecutivo

Este documento presenta la version final del Proyecto 4 de VeriLang. La solucion reutiliza la implementacion del Proyecto 3 en Rascal, que contiene parser, generador y sistema de tipos con TypePal, y la integra con una aplicacion Kotlin basada en el helper entregado para el proyecto. La aplicacion permite seleccionar archivos .vl, ejecutar el flujo del lenguaje, visualizar el resultado generado y reportar errores semanticos desde una interfaz grafica de escritorio.

La integracion se realizo mediante un modulo puente llamado RunnerJson.rsc. Este modulo invoca el parser, el checker y el generador de VeriLang, y devuelve un objeto JSON consumido por Kotlin. La aplicacion Kotlin ejecuta Rascal como subprocesso, interpreta el JSON con kotlinx.serialization y presenta el estado de parseo, chequeo de tipos, validacion semantica, salida generada y errores encontrados.

Tabla de contenido

1. Introduccion y objetivo
2. Requisitos abordados por la solucion
3. Estructura final del proyecto
4. Arquitectura de integracion Rascal-Kotlin
5. Componentes Rascal reutilizados
6. Modulo RunnerJson.rsc
7. Aplicacion Kotlin
8. Flujo de ejecucion
9. Configuracion y dependencias
10. Casos de prueba y evidencias
11. Como ejecutar el proyecto
12. Contenido de entrega
13. Conclusiones

1. Introduccion y objetivo

El objetivo del Proyecto 4 es tomar la implementacion existente de VeriLang en Rascal y ejecutarla desde el entorno de Kotlin. La solucion conserva la funcionalidad central del Proyecto 3 y agrega una capa de ejecucion grafica en Kotlin que actua como runtime e interfaz de usuario para el lenguaje.

El enunciado del proyecto establece que la implementacion previa de VeriLang debe reutilizarse, que el nuevo objetivo es Kotlin y que el resultado debe ser funcionalmente equivalente al Proyecto 3. Por esta razon, la implementacion final mantiene los modulos Rascal originales y agrega una comunicacion controlada entre Kotlin y Rascal.



ISIS-2111 PLE
Project 4
Fecha: May 8, 2026

Task 1. For the fourth part of the project, we will explore the capabilities of Rascal as a language workbench and the power of the definition of languages using formal grammars. To do this, your task is to reuse the implementation of VeriLang from the previous project, but now we will target a different language (i.e., not Java), Kotlin.

Your project should be functionally equivalent to project 3, but now run from the Kotlin runtime environment.

In addition to the project you will find a Kotlin helper file, that will help you in deploying your VeriLang to Kotlin.

Hand in a .zip file with the solution and document.

1

Figura 1. Enunciado del Proyecto 4: reutilizar VeriLang, apuntar a Kotlin y entregar solucion con documento.

2. Requisitos abordados por la solucion

Requisito	Implementacion en la solucion final
Reutilizar VeriLang del Proyecto 3	Se conservan Syntax.rsc, AST.rsc, Parser.rsc, Implode.rsc, Generator.rsc, Checker.rsc, Main.rsc y Plugin.rsc.
Ejecutar desde Kotlin	Se integra una aplicacion de escritorio en kotlin-app/ usando Compose Desktop.
Mantener equivalencia funcional	Desde Kotlin se puede parsear, chequear, reportar errores y mostrar la salida generada por VeriLang.
Usar el helper Kotlin	El helper se adapta mediante Main.kt, MainWindow.kt, LangService.kt y RunResult.kt.
Entregar solucion ejecutable	Se incluye rascal-shell-stable.jar y Gradle Wrapper para ejecutar la aplicacion sin depender de Gradle global.

3. Estructura final del proyecto

La raíz del proyecto contiene la implementacion Rascal original, la aplicacion Kotlin, los archivos de prueba y los archivos de configuracion necesarios para ejecutar la entrega final.

```

proyecto_rascal/
  META-INF/
    RASCAL.MF
  src/main/rascal/
    Syntax.rsc
    AST.rsc
    Parser.rsc
    Implode.rsc
    Generator.rsc
    Checker.rsc
    Main.rsc
    Plugin.rsc
    RunnerJson.rsc
  instance/
    spec_types.vl
    spec_typed_space_good.vl
    spec_typed_space_quantifier_good.vl
    errors/
      spec_unknown_space_error.vl
      spec_typed_space_quantifier_type_error.vl
    ...
  kotlin-app/
    build.gradle.kts
    settings.gradle.kts
    gradlew
    gradlew.bat
    gradle/wrapper/
  src/main/kotlin/milang/
    Main.kt
    model/RunResult.kt
    service/LangService.kt
    ui/MainWindow.kt
  pom.xml
  rascal-shell-stable.jar
  
```

Archivo o carpeta	Funcion
src/main/rascal/	Contiene la implementacion principal de VeriLang en Rascal.
RunnerJson.rsc	Puente de ejecucion entre Kotlin y los modulos Rascal.
kotlin-app/	Aplicacion Kotlin/Compose que permite seleccionar y ejecutar archivos .vl.
rascal-shell-stable.jar	Jar usado por Kotlin para abrir un proceso Rascal.
instance/	Programas VeriLang validos e invalidos usados en las pruebas.
pom.xml	Dependencias Maven de Rascal y TypePal.
.gitignore	Excluye artefactos generados como target/, build/, bin/, .gradle/ y archivos .lock.

4. Arquitectura de integracion Rascal-Kotlin

La arquitectura mantiene separadas las responsabilidades del lenguaje y de la interfaz. Rascal conserva la definicion y validacion del lenguaje, mientras que Kotlin actua como runtime grafico y consumidor del resultado JSON.

```

Usuario selecciona archivo .vl
    |
    v
MainWindow.kt
    |
    v
LangService.kt
    |
    v
rascal-shell-stable.jar
    |
    v
RunnerJson.rsc
    |
    +--> Parser.rsc
    +--> Checker.rsc + TypePal
    +--> Generator.rsc
    |
    v
JSON de resultado
    |
    v
RunResult.kt
    |
    v
Interfaz Kotlin: Parse / Types / Semantica / Output / Errores
  
```

El intercambio entre Rascal y Kotlin se hizo mediante JSON para evitar acoplar directamente la UI a estructuras internas de Rascal. Esta decision permite mostrar los resultados de forma uniforme, tanto en casos correctos como en casos con errores semanticos.

5. Componentes Rascal reutilizados

La base Rascal corresponde a VeriLang del Proyecto 3. Esta base incluye parser, AST, generador y checker con TypePal. El lenguaje reconoce modulos, espacios, variables, operadores, reglas, expresiones, cuantificadores, tipos primitivos y tipos definidos por el usuario.

Componente	Responsabilidad
Syntax.rsc	Define la gramatica concreta de VeriLang.
AST.rsc	Define la sintaxis abstracta usada por generador y checker.
Parser.rsc	Expone parseMainModule para leer archivos .vl.
Implode.rsc	Transforma el arbol concreto en AST.

```
[INFO] pom.xml: Rascal version is 0.42.7
[INFO] pom.xml: Project root is [file:///C:/Users/Roni/projecto_rascal/]
[INFO] pom.xml: Bin folder is [file:///C:/Users/Roni/projecto_rascal/target/classes]
[INFO] pom.xml: Source module path is:
  - [std:///]
  - [mvn://org.rascalml1--typepal--0.8.6/src]
  - [file:///C:/Users/Roni/projecto_rascal/src/main/rascal]
  - [jar+file:///C:/Users/Roni/.vscode/extensions/usethesource.rascalml1-0.13.5/assets/jars/rascal-lsp.jar!library]
[INFO] pom.xml: Library module (and classes) path is:
  - [file:///C:/Users/Roni/.vscode/extensions/usethesource.rascalml1-0.13.5/assets/jars/rascal.jar]
  - [mvn://org.rascalml1--typepal--0.8.6]
  - [mvn://junit--junit--4.13.1]
  - [mvn://org.hamcrest--hamcrest-core--1.3]
  - [file:///C:/Users/Roni/.vscode/extensions/usethesource.rascalml1-0.13.5/assets/jars/rascal-lsp.jar]
rascal>import RunnerJson;
ok
rascal>RunnerJson:main(["instance/spec_types.v1"]);
{"success":true,"module":"Verilang","parseOk":true,"typeCheckOk":true,"semanticOk":true,"typeErrors":[],"semanticErrors":[],"output":["defmodule TypeExamples\n\nndefspace Person endndefspace Group : Person endndefvar age : int otherAge : int active : bool verified : bool score : real otherScore : real name : string\notherName : string initial : char p : Person q : Person g : Group endndefoperator samePerson : Person -> Person -> bool endndefoperator belongsTo : Person -> Group -> bool endndefoperator combinesScores : real -> real -> real endndefexpression age = otherAge endndefexpression active and verified\nendndefexpression score = otherScore endndefexpression name = otherName endndefexpression initial = otherInitial endndefexpression p = q endndefexpression\np belongsTo g endndefexpression p samePerson q endndefexpression (forall member in Group . member belongsTo g) end\nend","error":"","codigoFormateado":"defmodule TypeExamples\n\nndefspace Person endndefspace Group : Person endndefvar age : int otherAge : int active : bool verified : bool score : real otherScore : real\nname : string otherName : string initial : char p : Person q : Person g : Group endndefoperator samePerson : Person -> Person -> bool\nendndefoperator belongsTo : Person -> Group -> bool endndefoperator combinesScores : real -> real -> real endndefexpression age = otherAge endndefexpression\nactive and verified endndefexpression score = otherScore endndefexpression name = otherName endndefexpression initial = otherInitial endndefexpression p =\nq endndefexpression p belongsTo g endndefexpression p samePerson q endndefexpression (forall member in Group . member belongsTo g) end\nend","resumen":"Programa Verilang parseado, chequeado y generado correctamente."}]
ok
rascal>
```

6. Modulo RunnerJson.rsc

```
import RunnerJson;
RunnerJson::main(["instance/spec_types.vl"]);

// Resultado representativo:
{
  "success": true,
  "module": "VeriLang",
  "parseOk": true,
  "typeCheckOk": true,
  "semanticOk": true,
  "typeErrors": [],
  "semanticErrors": [],
  "output": ["..."],
  "error": "",
  "codigoFormateado": "...",
}
```

```
"resumen": "Programa VeriLang parseado, chequeado y generado correctamente."
```

Quando el checker encuentra errores, el JSON conserva el parseo exitoso y marca la validacion de tipos y semantica como fallida. Esto permite que la interfaz distinga entre errores de lectura, errores de tipos y errores semanticos.

```
RunnerJson::main(["instance/errors/spec_unknown_space_error.v1"]);
```

```
// Resultado representativo:
```

```
{
  "success": false,
  "parseOk": true,
  "typeCheckOk": false,
  "semanticOk": false,
  "typeErrors": ["error("Undefined space `Person`, ...)"]
}
```

Figura 3. Ejecucion de RunnerJson desde la consola Rascal con salida JSON valida.

7. Aplicacion Kotlin

La aplicacion Kotlin se ubica en kotlin-app/. Esta aplicacion usa Compose Desktop para presentar una ventana grafica con un campo de ruta, boton de busqueda, boton de ejecucion y panel de resultados.

Archivo Kotlin	Funcion
Main.kt	Crea la ventana principal con titulo VeriLang.
MainWindow.kt	Construye la interfaz grafica, el selector de archivos .vl y el panel de resultados.
LangService.kt	Ejecuta Rascal como subprocesso y envia comandos a RunnerJson.rsc.
RunResult.kt	Define la estructura de datos que representa el JSON recibido desde Rascal.

La interfaz final muestra el nombre VeriLang y filtra archivos con extension .vl. Los indicadores de estado muestran Parse, Types y Semantica, cada uno con estado OK o FAIL.

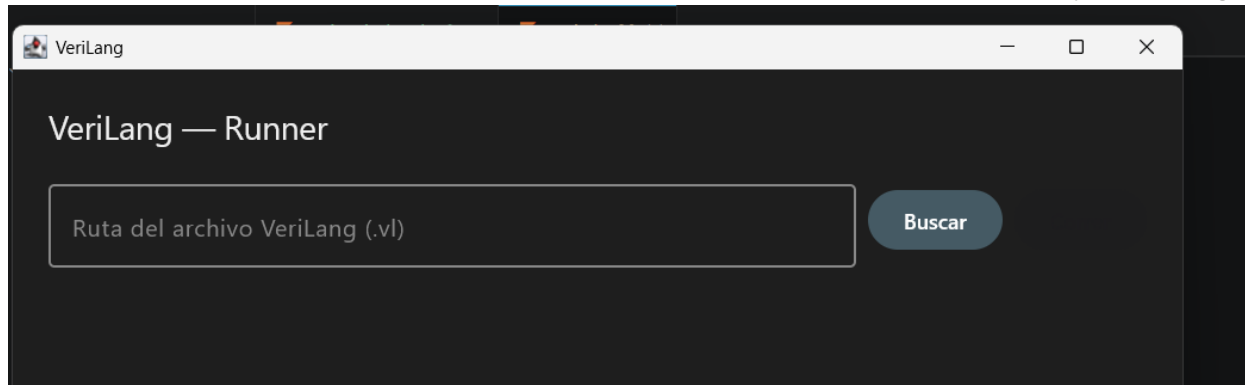


Figura 4. Interfaz final de la aplicacion Kotlin con titulo VeriLang y selector de archivos .vl.

8. Flujo de ejecucion

El flujo completo de ejecucion desde Kotlin es el siguiente:

Paso	Descripcion
1	El usuario selecciona un archivo VeriLang .vl desde la interfaz.
2	MainWindow.kt llama a LangService.run con la ruta del archivo.
3	LangService.kt abre rascal-shell-stable.jar como proceso Java.
4	El servicio escribe en stdin los comandos <code>import RunnerJson;</code> y <code>RunnerJson::main([...])</code> .
5	RunnerJson.rsc ejecuta <code>parseMainModule</code> , <code>modulesTModelFromTree</code> y <code>generator</code> .
6	Rascal imprime un JSON con el estado de ejecucion y los mensajes producidos.
7	Kotlin extrae el JSON, lo decodifica como <code>RunResult</code> y actualiza la interfaz.

Este flujo conserva el comportamiento de la ejecucion por terminal, pero lo expone desde el runtime de Kotlin. Si el programa es valido, se muestra el codigo generado; si contiene errores semanticos, la interfaz presenta los mensajes correspondientes.

9. Configuracion y dependencias

El proyecto usa Maven para las dependencias Rascal y TypePal, y Gradle para construir y ejecutar la aplicacion Kotlin. La entrega incluye Gradle Wrapper para facilitar la ejecucion desde otro entorno.

```
pom.xml
org.rascalimpl:rascal
org.rascalimpl:typepal:0.8.6

kotlin-app/build.gradle.kts
kotlin("jvm") version "1.9.22"
org.jetbrains.compose version "1.6.0"
kotlinx-serialization-json
kotlinx-coroutines
```


kotlin-app/gradlew.bat

ejecuta la aplicacion sin requerir Gradle global

El archivo rascal-shell-stable.jar queda en la raiz del proyecto. LangService.kt lo localiza desde la aplicacion Kotlin y lo usa para abrir la consola Rascal en modo subprocesso.

```
PS C:\Users\Roni\proyecto_rascal> Copy-Item "$env:USERPROFILE\.vscode\extensions\usethesource.rascalimpl-0.13.5\assets\jars\rascal.jar" "C:\Users\Roni\proyecto_rascal\rascal-shell-stable.jar"
PS C:\Users\Roni\proyecto_rascal> dir C:\Users\Roni\proyecto_rascal\rascal-shell-stable.jar

Directory: C:\Users\Roni\proyecto_rascal

Mode                LastWriteTime         Length Name
----                -
-a-----         22/04/2026      3:22      82162555 rascal-shell-stable.jar
```

Figura 5. Archivo rascal-shell-stable.jar ubicado en la raiz del proyecto despues de copiar rascal.jar.

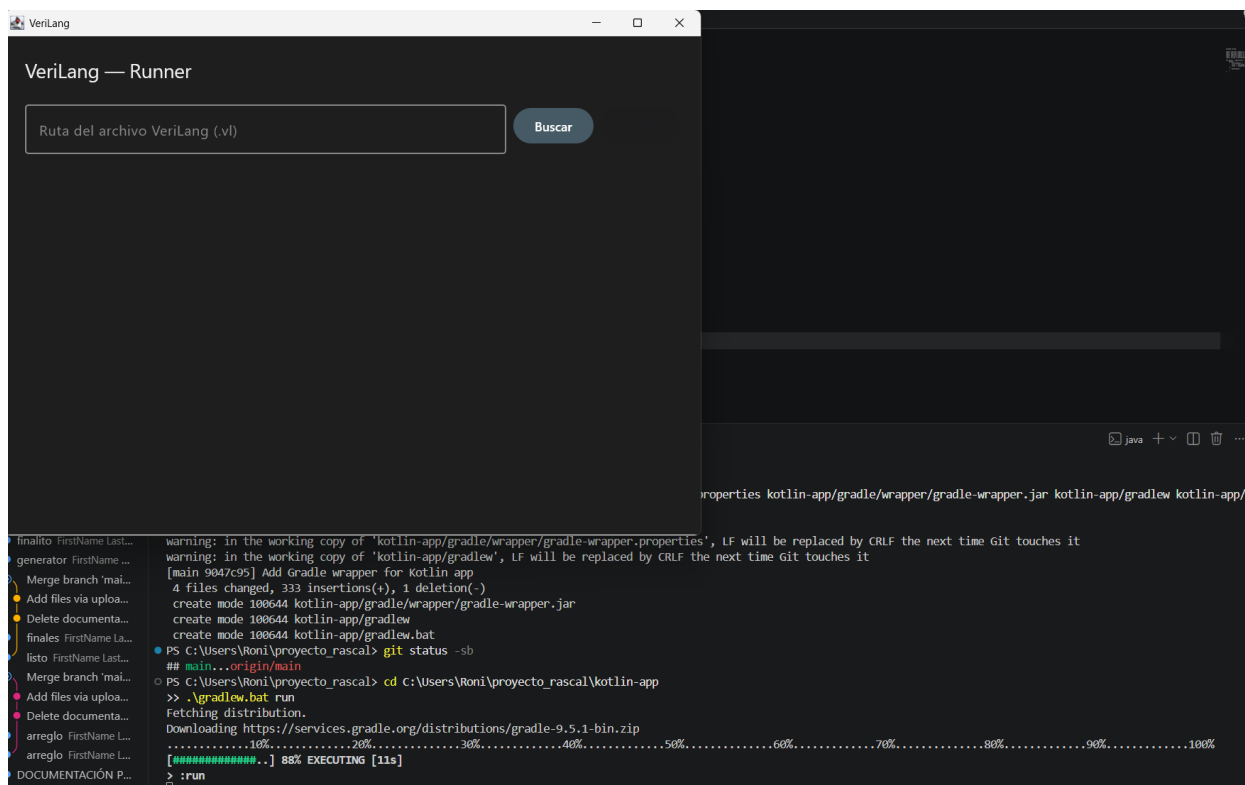


Figura 6. Ejecucion de la aplicacion con Gradle Wrapper mediante .\gradlew.bat run.

10. Casos de prueba y evidencias

La solucion se valido con pruebas de ejecucion directa en Rascal, pruebas del puente JSON y pruebas desde la interfaz Kotlin. Los casos seleccionados demuestran tanto ejecuciones correctas como reportes de error.

Prueba	Entrada	Resultado observado
Rascal valido	main() con instance/spec_types.vl	Parser OK, checker OK, generacion OK, int: 0.
Rascal invalido	main(cwd:///instance/errors/spec_unknown_space_error.vl)	Error Undefined space `Person`, no genera codigo, int: 1.
RunnerJson valido	RunnerJson::main(["instance/	JSON con success=true, parseOk=true,

	spec_types.vl"]])	typeCheckOk=true, semanticOk=true.
RunnerJson invalido	RunnerJson::main(["instance/errors/spec_unknown_space_error.vl"]])	JSON con success=false, typeCheckOk=false, semanticOk=false y Undefined space `Person`.
Kotlin valido	Seleccionar instance/spec_types.vl en la UI	Parse OK, Types OK, Semantica OK y salida generada.
Kotlin invalido	Seleccionar instance/errors/spec_unknown_space_error.vl en la UI	Parse OK, Types FAIL, Semantica FAIL y error Undefined space `Person`.

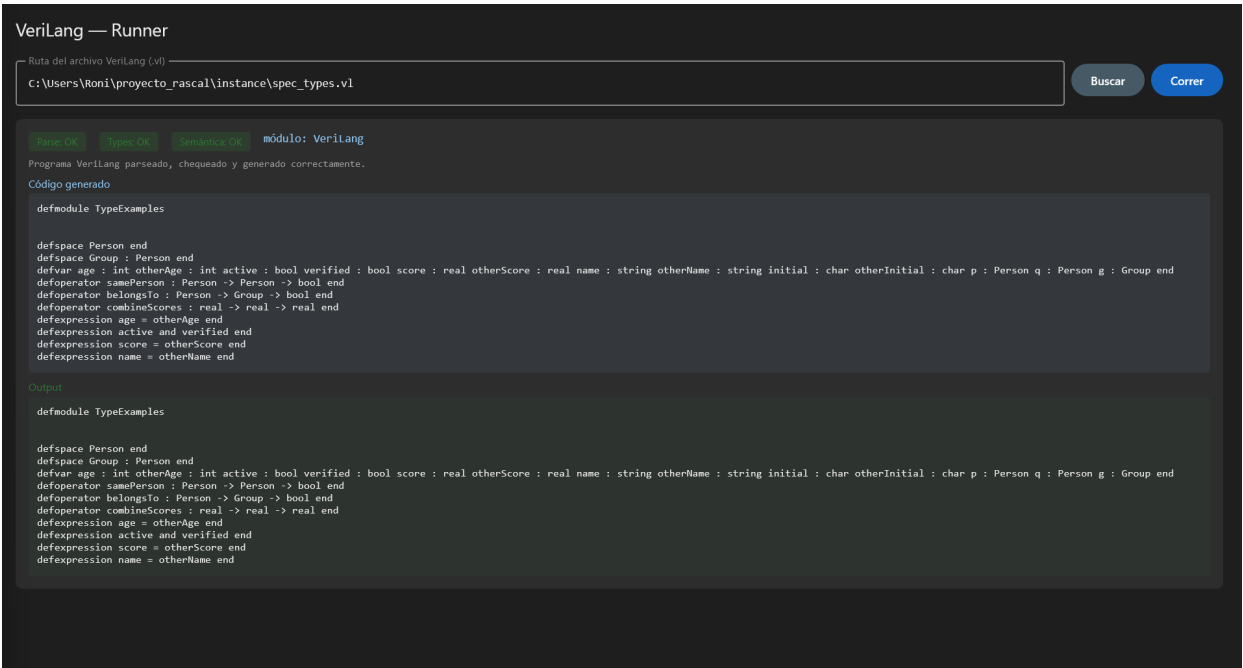


Figura 7. Ejecucion final valida desde la interfaz VeriLang, mostrando codigo generado y output.

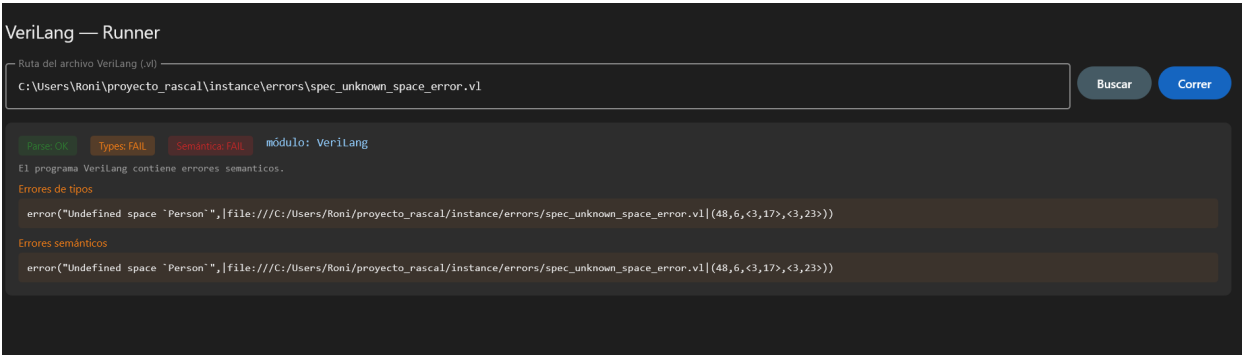


Figura 8. Ejecucion final invalida desde la interfaz VeriLang, mostrando Types FAIL y Semantica FAIL.

11. Como ejecutar el proyecto

Para ejecutar la parte Rascal, se abre una terminal Rascal ubicada en la raiz del proyecto y se importan los modulos correspondientes.

```
import Main;
main();
main(| cwd:///instance/errors/spec_unknown_space_error.vl|);
```

```
import RunnerJson;
RunnerJson::main(["instance/spec_types.vl"]);
RunnerJson::main(["instance/errors/spec_unknown_space_error.vl"]);
```

Para ejecutar la aplicacion Kotlin, se usa el Gradle Wrapper incluido en la carpeta kotlin-app.

```
cd kotlin-app
.\gradlew.bat run
```

Una vez abierta la ventana, se selecciona un archivo .vl. Para un programa valido, la interfaz presenta estados OK y salida generada. Para un programa invalido, presenta el error semantico devuelto por TypePal.

12. Contenido de entrega

Elemento	Descripcion
Proyecto Rascal	Modulos de VeriLang en src/main/rascal.
Aplicacion Kotlin	kotlin-app con Compose Desktop, Gradle Wrapper y codigo fuente.
RunnerJson.rsc	Modulo puente entre Rascal y Kotlin.
rascal-shell-stable.jar	Jar usado por la aplicacion para ejecutar Rascal.
Casos de prueba	Archivos .vl validos e invalidos dentro de instance/.
Documento final	Explicacion de arquitectura, ejecucion y evidencias.

Los directorios generados por ejecucion o compilacion, como target/, build/, bin/ y .gradle/, no forman parte del contenido fuente de la solucion.

13. Conclusiones

La version final del Proyecto 4 ejecuta VeriLang desde el entorno de Kotlin manteniendo la funcionalidad del Proyecto 3. El parser, el checker con TypePal y el generador permanecen implementados en Rascal, mientras que Kotlin proporciona una interfaz de ejecucion y visualizacion de resultados.

El modulo RunnerJson.rsc establece una frontera clara entre ambos entornos. Rascal produce un resultado JSON y Kotlin lo interpreta para mostrar estados, errores y salida generada. Esta integracion permite comprobar programas correctos y programas con errores semanticos desde una aplicacion de escritorio.

Las evidencias incluidas muestran que la solucion parsea, chequea y genera correctamente programas validos, y que reporta errores semanticos esperados en programas invalidos. La entrega incluye tambien Gradle Wrapper y el jar de ejecucion Rascal, de forma que la aplicacion Kotlin pueda ejecutarse desde la estructura final del proyecto.